Compression d'images

Ce DM est à réaliser en C.

Préliminaires

Ce sujet s'inscrit dans la continuité du TP 18 sur l'algorithme des k-moyennes. Vous pouvez récupérer le fichier corrigé du TP 18 ou le finir. Dans tous le sujet, on suppose que l'on a une image que l'on veut compresser à l'aide d'une palette de k couleurs que l'on a déjà calculé. On reprend les types du TP à savoir :

```c
struct pixel{
    int rouge;
    int vert;
    int bleu;
};

typedef struct pixel pixel_t;

struct image{
    int hauteur;
    int longueur;
    pixel_t* tableau;
};

typedef struct image image_t;
```

On pourra utiliser la fonction `k_moyennes` du fichier `TP18.c` corrigé pour générer une palette de couleurs qui semble intéressante.

On remarque qu'après restriction de la palette de couleurs, les zones de l'image à la frontière entre deux couleurs ne sont pas très esthétiques. En effet, la cassure nette entre celles-ci est très visible.

FIGURE 1 – Un exemple d'artefact visuel pouvant apparaître.

On cherche donc dans ce sujet à rendre les transitions entre couleurs plus fluides tout en conservant la palette calculée au préalable. Pour cela, lorsque l'on remplace un pixel par un autre de la palette, on calcule l'erreur que l'on fait par cette approximation. Cette erreur est propagée sur les pixels voisins. Ainsi, lorsque l'on traitera ceux-ci, on aura tendance à choisir une couleur qui compense l'erreur précédente.

1 Propagation d'erreur

Question 1 : Écrire une fonction `pixel_t erreur(pixel_t a_remplacer, pixel_t representant)` qui, étant donné un pixel et le pixel de la palette qui le représentera, renvoie un pixel indiquant les erreurs liées au remplacement. Par exemple si le pixel gris (100, 100, 100) est remplacé par le pixel rouge (255, 0, 0), on renverra le pixel (-155, 100, 100).

Pour propager les erreurs, on utilise une distribution au pixels alentours. Par exemple, la matrice suivante indique la répartition de l'erreur sur les pixels voisins :

$$\begin{bmatrix} & * & \frac{7}{16} \\ \frac{3}{16} & \frac{5}{16} & \frac{1}{16} \end{bmatrix}$$

Le pixel actuellement traité est le pixel indiqué par $*$. Avec une telle distribution, 7/16ème de l'erreur obtenue sur le pixel traité est propagée au pixel directement à droite tandis que 5/16 est propagé à celui en dessous. On remarque que la somme des propagations est égale à 1. On représentera la distribution de dimension sous forme d'une matrice de flottants de taille $n \times m$. Le pixel traité sera à l'indice $(m-1)/2$ de la ligne 0.

On introduit donc le type suivant :

```
struct distribution{
    float** matrice;
    int n;
    int m;
};

typdef struct distribution distribution_t;
```

Question 2 : Écrire une fonction `pixel_t** matrice_erreurs(pixel_t erreur, distribution_t d)` qui, étant donnés une erreur à propager ainsi qu'une distribution de propagation, renvoie la matrice des erreurs propagées. La matrice renvoyée sera de la même dimension et indiquera pour chaque case du voisinage l'erreur qui s'y retrouve propagée.

Question 3 : Écrire une fonction `void propager(image_t img, int i, int j, pixel_t erreur, distribution_t d)` qui, étant donné une image, les coordonnées d'un pixel ayant été remplacé, l'erreur produite et une distribution, modifie l'image fournie en propageant les erreurs liées au remplacement. On veillera à traiter l'erreur devrait être propagée sur des pixels hors de l'image.

2 Corriger l'image

Question 4 : Écrire une fonction `void correction_unitaire(image_t img, int i, int j, distribution_t d, pixel_t* palette, int k)` qui, étant donnée une image et les coordonnées d'un pixel, le remplace par celui de la palette de k couleurs le plus proche. L'erreur engendrée sera propagée d'après la matrice de confusion.

Question 5 : On souhaite corriger les pixels depuis le coin supérieur gauche en travaillant ligne par ligne. Écrire une fonction `void correction_totale(image_t img, distribution_t d, pixel* palette, int k)` qui utilise la palette de k couleurs pour représenter l'image aussi fidèlement que possible d'après la matrice de propagation d'erreur.

3 Compresser

Question 6 : Écrire une fonction `void compresser_propagation(char* image_source, char* image_destination, int k, distribution_t d)` qui, étant donnée une image en format $P3$, la compresse en utilisant seulement k couleurs en appliquant la propagation d'erreur. L'image ainsi obtenue sera enregistrée dans le fichier `image_destination`. On pensera à utiliser ou à s'inspirer des questions précédentes ou des fonctions disponibles dans le corrigé du TP18.